FINANCIAL RISK PLATFORM

Detailed Data Model

Logical entities, relationships, versioning, lineage, security and extensibility

Purpose. Define a platform-wide logical data model for the lightweight but extensible financial-risk platform across five synchronized workstreams and four architectural layers.

Status. Working draft v0.1. This is a proposed logical model, not a confirmed client physical schema. Source attributes, jurisdiction, regulatory rules, product coverage and integrations remain subject to discovery.

Design principle. Preserve source provenance; map into versioned canonical contracts; keep historical calculation inputs immutable; trace each result to data, schema, mapping, assumptions, rules and engine versions.

Model area	Primary purpose	Workstream
Governance and security	Tenant scope, identity, permissions, classification and audit	WS1
Schema and contracts	Versioned schemas, source profiles, mappings and change proposals	WS1/WS2
Ingestion and quality	Imports, records, validation, quarantine, profiling and reconciliation	WS2/WS4
Financial domain	Parties, accounts, instruments, positions, cash flows, collateral, market/reference data	WS3
Rules and calculations	Rules, modules, runs, outputs, intermediate values and warnings	WS3/WS4
Operations and lineage	Scenarios, benchmarks, reports, workflow and audit	WS1/WS4
AI governance	AI requests, retrieval evidence, tool calls, proposals and evaluation	WS5

1. Architecture and modelling principles

• Use a modular relational core (e.g. PostgreSQL) for metadata, entities, configurations, job states and results; use object storage for original files and large artifacts.

• Keep source payloads and canonical records distinct. A source file is evidence of what arrived; canonical records represent interpretation under a schema and mapping version.

• Use immutable versions for schemas, mappings, instrument terms, rules, assumption sets, module versions and report definitions.

• Every calculation run pins the exact dataset snapshot and all configuration/model versions. Historic run inputs are never updated in place.

• Use stable internal UUIDs plus source-scoped external IDs. Do not assume client identifiers are globally unique or immutable.

• Use fixed-precision decimal for authoritative financial values, not binary floating point. Store currency/unit and as-of/effective dates explicitly.

• Use typed columns for stable, frequently queried fields and governed extension attributes for genuinely variable source-specific fields.

• Separate calculation definitions/configuration from results; preserve evidence for independent reconciliation and explanation.

• Security, masking, retention, access control and audit are cross-cutting model requirements, not UI-only features.

2. High-level relationship map

source_system → source_profile/source_contract → import_job/import_artifact → source_record → field_mapping/mapping_version → canonical_record/instrument_version/position_version → calculation_run (pinned inputs + rule_set + assumption_set + module_version) → intermediate_value and domain result tables → benchmark_result/report_instance. lineage_node/lineage_edge and audit_event preserve provenance and governance.

• One logical schema has many immutable versions; each import and run references a specific version.

• One mapping definition has many versions; each import records the exact mapping version applied.

• One instrument has many immutable term versions and many positions over time.

• One rule definition has many versions; a rule set contains explicit rule versions and precedence.

• One calculation run has many input references, outputs, intermediate values, warnings and lineage edges.

• One scenario references a baseline and stores proposed changes separately; it never mutates baseline configuration.

• AI proposals are separate from approved rules/configuration. Approval and activation are explicit state transitions.

3. Data dictionary and entity catalogue

All entities normally include created_at, updated_at, created_by and tenant_id where applicable. Immutable/versioned records also include status, version/effective dates and a content hash as appropriate. Physical columns should be finalized after client data samples and domain conventions are confirmed.

1. Governance, identity and security

Entity	Key field(s)	Logical key	Purpose / modelling notes
tenant	tenant_id	UUID PK	Organisation/security boundary; retain even if first demo is single-tenant.
user_account	user_id	UUID PK	Application identity mapped to an identity-provider subject; do not store passwords.
role	role_id	UUID PK	Viewer, analyst, approver, administrator or client-approved role.
permission	permission_id	UUID PK	Action/resource permission enforced server-side.
user_role	user_id + role_id	Composite PK/FK	User-to-role assignment, tenant-scoped where applicable.

Entity	Key field(s)	Logical key	Purpose / modelling notes
role_permission	role_id + permission_id	Composite PK/FK	Role-to-permission assignment.
data_classification	classification_id	UUID PK	Client-approved labels such as internal, confidential, restricted.
access_policy	policy_id	UUID PK	Policy for resource, action, scope, masking and allowed use.
audit_event	audit_event_id	UUID PK	Actor, action, resource, timestamp, outcome, correlation ID and safe metadata.

2. Schema registry and source contracts

Entity	Key field(s)	Logical key	Purpose / modelling notes
schema_definition	schema_id	UUID PK	Logical contract such as instrument, position, cash flow or market observation.
schema_version	schema_version_id	UUID PK	Immutable schema version, contract payload, status, effective date and compatibility level.
schema_field	schema_field_id	UUID PK	Field name/type, requiredness, unit, scale, sensitivity, validation profile and extension eligibility.
schema_change_proposal	schema_change_id	UUID PK	Schema diff, impact analysis, regression evidence, approval and rollback reference.
schema_dependency	schema_id + dependent ref	Composite/logical FK	Links schemas to mappings, rules, modules, reports and integrations.
source_system	source_system_id	UUID PK	Warehouse, file drop, internal application or future feed/API.
source_profile	source_profile_id	UUID PK	Format, encoding, delimiter, sheet/table, cadence, owner and provisional/approved state.
source_contract	source_contract_id	UUID PK	Expected incoming shape and version; can remain provisional until client samples arrive.
field_mapping	mapping_id	UUID PK	Logical source-to-canonical mapping definition.
mapping_version	mapping_version_id	UUID PK	Immutable mapping version used by an import/run.
mapping_rule	mapping_rule_id	UUID PK	Source selector, target field, transform, parameters, null/default policy and validation.

3. Ingestion, validation and data quality

Entity	Key field(s)	Logical key	Purpose / modelling notes
import_job	import_id	UUID PK	One ingestion attempt with source profile, status, schema/mapping refs, actor, timestamps and correlation ID.
import_artifact	artifact_id	UUID PK	Original file/object reference, MIME type, size, checksum, classification and retention state.
import_batch	batch_id	UUID PK	Logical batch/partition and source batch identifier with row counts.
source_record	source_record_id	UUID PK	Stable identity for a row/object, with source locator and safe provenance metadata.
canonical_record	canonical_record_id	UUID PK	Normalized record under a specific canonical schema version.
record_lineage	lineage_id	UUID PK	Links canonical records to source records, mapping version and transformation/run.
validation_profile	validation_profile_id	UUID PK	Versioned checks by schema, source or module.
validation_issue	issue_id	UUID PK	Issue code, severity, field/record ref, message, disposition and status; avoid sensitive values in messages.
quarantine_item	quarantine_id	UUID PK	Rejected/held record reference, reason, owner, status and retry/disposition history.
reconciliation_run	reconciliation_id	UUID PK	Counts/totals/keys comparison, tolerance policy and outcome.

Entity	Key field(s)	Logical key	Purpose / modelling notes
reconciliation_item	reconciliation_item_id	UUID PK	Individual check/record variance, tolerance and resolution.
data_profile_snapshot	profile_id	UUID PK	Observed types, null rates, uniqueness, ranges and distribution summaries.
data_drift_event	drift_event_id	UUID PK	Observed shape/distribution change; creates a review event, not automatic schema mutation.

4. Financial domain and canonical model

Entity	Key field(s)	Logical key	Purpose / modelling notes
legal_entity	legal_entity_id	UUID PK	Bank/business legal entity; jurisdiction and reporting scope require confirmation.
counterparty	counterparty_id	UUID PK	Counterparty reference with source-scoped external IDs and classification.
account	account_id	UUID PK	Account/ledger reference, owning legal entity and currency.
instrument_type	instrument_type_id	UUID PK	Versioned instrument class/template such as deposit, loan, bond, security or derivative.
instrument	instrument_id	UUID PK	Stable canonical identity, type, issuer/counterparty, currency and product family.
instrument_version	instrument_version_id	UUID PK	Immutable effective-dated contractual terms snapshot.
instrument_attribute_definition	attribute_id	UUID PK	Typed, governed extension attribute scoped to instrument type/schema version.
instrument_attribute_value	instrument_version_id + attribute_id	Composite PK/FK	Typed extension value; do not use for frequently queried core fields.
position	position_id	UUID PK	Holding/exposure at a point in time, instrument, account, quantity, balances, currency and valuation date.
position_version	position_version_id	UUID PK	Immutable position snapshot for reproducible calculations.
contractual_term	term_id	UUID PK	Principal, rate type, margin, reset frequency, day-count, amortisation and schedule reference.
cashflow_schedule	schedule_id	UUID PK	Versioned schedule definition, frequency, calendar and convention.
cashflow_event	cashflow_id	UUID PK	Cash-flow event with due date, amount, currency, type and contractual/behavioural origin.
cashflow_component	cashflow_component_id	UUID PK	Principal, interest, fee, repayment or other component with derivation lineage.
collateral_asset	collateral_id	UUID PK	Collateral asset, owner, type, valuation, currency, eligibility and encumbrance attributes.
collateral_link	collateral_link_id	UUID PK	Collateral-to-exposure allocation, amount, effective period and allocation constraints.
market_observation	market_observation_id	UUID PK	Price/rate/curve/FX/spread value, unit, observed_at, source, quality and version.
reference_data_item	reference_item_id	UUID PK	Currencies, calendars, rating categories, instrument codes or controlled vocabularies.
assumption_set	assumption_set_id	UUID PK	Versioned assumptions with provenance, owner, effective dates and approval state.

5. Rules, calculation engine and results

Entity	Key field(s)	Logical key	Purpose / modelling notes
rule_definition	rule_id	UUID PK	Stable rule identity, category, target metric, parameter schema and owner.
rule_version	rule_version_id	UUID PK	Immutable rule parameters/formula reference, effective dates, status and approval.

Entity	Key field(s)	Logical key	Purpose / modelling notes
rule_set	rule_set_id	UUID PK	Package of rule versions for module, product, jurisdiction or reporting context.
rule_set_member	rule_set_id + rule_version_id	Composite PK/FK	Explicit rule membership and precedence.
calculation_module	module_id	UUID PK	Module identity: cash flow, maturity ladder, LCR/HQLA, ALM/IRRBB, capital or collateral.
module_version	module_version_id	UUID PK	Engine/module version and supported input/output schema versions.
calculation_run	run_id	UUID PK	Immutable execution pinned to dataset snapshot, schema/mapping, rules, assumptions and module/engine versions.
calculation_input_ref	run_id + input ref	Composite/logical FK	Exact input snapshots/configurations consumed by a run.
cashflow_result	cashflow_result_id	UUID PK	Run-specific generated cash flow, distinct from source/contractual cash flow.
maturity_bucket_definition	bucket_definition_id	UUID PK	Versioned bucket boundaries and names; domain expert must approve boundaries.
liquidity_gap_result	liquidity_gap_result_id	UUID PK	Run/bucket inflows, outflows, net gap and cumulative position.
hqla_asset_result	hqla_result_id	UUID PK	Eligibility, value, haircut, cap/restriction and inclusion/exclusion reason for the run.
lcr_result	lcr_result_id	UUID PK	HQLA, net outflows, ratio, currency/scope and rule version.
capital_result	capital_result_id	UUID PK	Capital components, exposure/RWA references and ratios; scope requires regulatory confirmation.
collateral_result	collateral_result_id	UUID PK	Run-specific valuation, allocation, eligibility and encumbrance treatment.
intermediate_value	intermediate_value_id	UUID PK	Named calculation intermediate, value/unit/dimensions, formula/rule ref and lineage.
calculation_warning	warning_id	UUID PK	Warning/exception by run/entity/metric, severity, code and resolution status.

6. Operations, scenarios, lineage and reporting

Entity	Key field(s)	Logical key	Purpose / modelling notes
scenario	scenario_id	UUID PK	Scenario overlay over baseline configuration; never mutates approved baseline.
scenario_change	scenario_change_id	UUID PK	Proposed rule/assumption/input change, old/new refs, rationale and approval state.
scenario_run	scenario_run_id	UUID PK	Baseline and proposed run references for comparison on the same input snapshot.
lineage_node	lineage_node_id	UUID PK	Source, canonical record, transformation, rule, intermediate or output node.
lineage_edge	lineage_edge_id	UUID PK	Directed provenance relationship between lineage nodes.
benchmark_case	benchmark_case_id	UUID PK	Approved expected case, source, owner, tolerance policy and validity.
benchmark_result	benchmark_result_id	UUID PK	Actual-versus-expected comparison at instrument/component/bucket/metric level.
report_definition	report_definition_id	UUID PK	Versioned report layout, fields, aggregation and access policy.
report_instance	report_instance_id	UUID PK	Generated report bound to run and report-definition version.
workflow_job	job_id	UUID PK	Long-running import/calculation/reconciliation/report/AI job with state and retries.
review_task	review_task_id	UUID PK	Approval/exception item assigned to user/role with decision and reason.
event_outbox	event_id	UUID PK	Reliable internal event record for job status, approvals and change events.

7. AI/GenAI and model governance

Entity	Key field(s)	Logical key	Purpose / modelling notes
ai_request	ai_request_id	UUID PK	Task, requester, permitted data scope, status, model/provider/version and prompt-template version.
ai_retrieval_reference	retrieval_ref_id	UUID PK	Document/schema/run/lineage source reference with access check and source version.
ai_tool_call	tool_call_id	UUID PK	Allowlisted tool/API call, redacted arguments or hash, authorization result and outcome.
ai_proposal	proposal_id	UUID PK	Structured mapping/rule/scenario proposal, validation result, reviewer and approval state.
ai_evaluation_run	evaluation_run_id	UUID PK	Evaluation dataset/test suite, model/prompt version, scores, failure cases and disposition.
ai_feedback	feedback_id	UUID PK	Feedback reason and linked request; avoid unnecessary sensitive free text.

4. Candidate canonical financial attributes

Candidate fields only. A source mapping may leave fields unavailable, derive them through an approved transform, or route a record to review. Do not fabricate missing financial terms.

Entity	Candidate attribute	Type	Meaning	Rule/caveat
Instrument	instrument_id	UUID	Internal stable ID	Generated internally
Instrument	source_system_id + source_instrument_id	UUID/string	External identity	External key is scoped by source system
Instrument	instrument_type_id	UUID	Controlled type	Must resolve to an approved type/template
Instrument	legal_entity_id / counterparty_id	UUID nullable	Relationships	Nullable only if domain rules allow
Instrument	currency_code	String	Currency	Validate against controlled reference data
Instrument	contractual_start_date / maturity_date	Date nullable	Contract dates	Semantics depend on instrument type
Instrument	rate_type / reference_rate / spread	Enum/string/decimal nullable	Rate terms	Applicability depends on instrument type
Instrument	day_count_convention / payment_frequency	Enum/string nullable	Conventions	Controlled values and effective versions
Position	position_id / instrument_id	UUID	Identity/relationship	Position is a holding snapshot, not instrument master
Position	as_of_date / valuation_date	Date	Temporal scope	Required according to source and calculation
Position	nominal_amount / carrying_value / market_value	Decimal nullable	Amounts	Keep as distinct concepts; never overload one balance field
Position	currency_code / quantity	String / Decimal nullable	Units	Currency and unit explicit
Cash flow	cashflow_id / instrument_version_id	UUID	Lineage	Contractual and behavioural origins must be distinguished
Cash flow	due_date / amount / currency_code	Date / Decimal / string	Value/timing	Record generation method/version when generated
Market observation	observation_type / value / unit	Enum/string / Decimal / string	Observation	Include source, observed_at, quality and staleness
Collateral	collateral_id / secured_exposure_id	UUID	Relationship	Use a link/allocation entity for many-to-many relations
Versioned records	valid_from / valid_to / status	Timestamp/date / enum	Effective dating	Prevent ambiguous overlapping active versions
Source-derived records	source_record_id / import_id / mapping_version_id	UUID	Provenance	Preserve source locator and transformation lineage

5. Flexible source shape strategy

- Source adapters normalize transport/file details into import jobs and source records without assuming a fixed client schema.
- Until client samples arrive, use provisional source contracts and synthetic fixtures clearly labelled illustrative.
- Profile column names, observed types, null rates, uniqueness, ranges, date formats, currency patterns and distribution summaries.
- Mapping suggestions require human review and versioning before calculation use.
- Unknown fields are retained in protected artifact references or governed extension structures subject to classification and retention policy; never silently discard them.
- Schema drift creates a review event and impact report, not an automatic schema or financial-interpretation change.

• Define an adapter contract so warehouse/API/feed connectors can be added without changing the core domain model.

6. Data lifecycle and change governance

Step	Required behaviour
1. Register	Create source profile and provisional/approved source contract; assign owner and classification.
2. Receive	Create import job, artifact metadata, checksum, batch and correlation ID; enforce file/security controls.
3. Profile	Inspect structure and quality; produce suggestions only, not schema mutations.
4. Map	Choose canonical schema and mapping versions; validate transforms and required fields.
5. Validate	Run schema/domain checks; create field-level issues/quarantine references; reconcile counts/totals.
6. Canonicalize	Create normalized records with source lineage and exact schema/mapping references.
7. Configure	Select approved assumptions, rule set and module version; isolate scenario changes from baseline.
8. Calculate	Create immutable run pinned to exact inputs/configuration; persist outputs, intermediate values and warnings.
9. Verify	Compare against approved benchmark cases at instrument/component/bucket/final metric levels.
10. Review/publish	Apply authorization/approval policy; generate reports and audit consequential actions.
11. Change	Propose schema/mapping/rule change; assess dependencies, test, approve, activate and retain rollback reference.

7. Calculation reproducibility and lineage

• Each calculation_run references dataset_snapshot_id (or immutable input manifest), schema_version_id, mapping_version_id(s), rule_set/version, assumption_set_id, module_version_id and engine build/version.

• Persist start/end time, status, warnings, request/correlation ID and content hashes for important input/configuration manifests.

• Outputs carry metric code, value, unit/currency, dimensions, as-of date, run ID and lineage references.

• Trace final metric → component/bucket → intermediate values/rules → canonical records → source records/import artifact.

• Numerical tolerances are defined per metric by domain experts; no global tolerance is assumed.

• AI explanations read stored outputs and lineage; they never generate authoritative financial metrics.

8. Security and privacy controls

• Classify fields/artifacts and apply least privilege and tenant/portfolio/legal-entity scope where required.

• Keep identity-provider secrets and encryption keys outside application data; store only key references.

• Apply masking/redaction to logs, errors, AI retrieval, exports and UI according to approved policy.

• Audit schema/mapping/rule changes, approvals, calculation runs, exports and AI tool calls.

• Define retention/deletion for raw artifacts, derived records, AI requests and logs.

• Use synthetic data for demos unless client data use is explicitly approved; separate test and production environments.

• Enforce authorization server-side for data, lineage, reports and AI retrieval/tool calls.

9. Suggested status enums

Field	Proposed values
schema_version.status	DRAFT, IN_REVIEW, APPROVED, ACTIVE, DEPRECATED, RETIRED
source_contract.status	PROVISIONAL, IN_REVIEW, APPROVED, SUPERSEDED
import_job.status	CREATED, RECEIVED, PROFILING, MAPPING_REQUIRED, VALIDATING, COMPLETED, COMPLETED_WITH_WARNINGS, FAILED, CANCELLED
validation_issue.severity	INFO, WARNING, ERROR, BLOCKER
quarantine_item.status	OPEN, UNDER_REVIEW, REJECTED, CORRECTED, RETRY_READY, RESOLVED
rule_version.status	DRAFT, IN_REVIEW, APPROVED, ACTIVE, SUPERSEDED, RETIRED
calculation_run.status	QUEUED, RUNNING, SUCCEEDED, SUCCEEDED_WITH_WARNINGS, FAILED, CANCELLED, BLOCKED
scenario.status	DRAFT, IN_REVIEW, APPROVED_FOR_TEST, RUNNING, TESTED, REJECTED, CLOSED
review_task.decision	PENDING, APPROVED, REJECTED, NEEDS_CHANGES, ESCALATED
ai_proposal.status	DRAFT, VALIDATED, IN_REVIEW, APPROVED_FOR_SCENARIO, REJECTED, EXPIRED

10. Example import and calculation manifest

Illustrative JSON only. Final field names and schemas must be aligned in OpenAPI; no real client schema is implied.

```
{
  "import_id": "imp_...",
  "source_profile_id": "src_profile_...",
  "source_contract_version": "provisional-0.1",
  "schema_version_id": "schema_ver_...",
  "mapping_version_id": "mapping_ver_...",
  "artifact": {
    "sha256": "...",
    "storage_ref": "protected-object-reference",
    "classification": "confidential"
  },
  "quality": {
    "accepted_records": 980,
    "rejected_records": 20,
    "warnings": 7,
    "reconciliation_status": "REVIEW_REQUIRED"
  },
  "calculation_manifest": {
    "dataset_snapshot_id": "snapshot_...",
    "rule_set_version_id": "rule_set_ver_...",
    "assumption_set_id": "assumptions_...",
    "module_version_id": "module_ver_...",
    "engine_version": "build_...",
    "run_id": "run_..."
  }
}
```

11. Deliberately unresolved decisions

• Actual source files, feed protocols, attributes, source-of-truth precedence and ownership.

• Legal entities, jurisdiction, regulatory framework/effective dates, required reports and calculation conventions.

- Instrument coverage, behavioural models, market data/rating providers and licensing.
- Physical table naming, partitioning, retention, data volumes, performance targets, deployment and recovery objectives.
- Exact LCR/HQLA, ALM/IRRBB, capital and collateral rules and numerical tolerances.
- Whether stress testing, FTP, impairment, investment management or other modules are in scope.

12. Acceptance checklist

- Every entity has purpose, stable key, ownership, sensitivity classification and lifecycle.
- Source schema, canonical schema and mappings are separate, versioned objects.
- Unknown source fields can be profiled/reviewed without silently mutating the canonical schema.
- Historic instrument terms, rules, mappings and assumptions used by a run remain immutable and referenceable.
- Calculation output traces to source records and exact transformation/rule/engine versions.
- Authorization and audit are enforced server-side across data, calculations, reports and AI.
- Schema/rule changes have impact analysis, compatibility checks, regression evidence, approval and recovery path.
- Domain experts approve canonical fields, calculation specifications, benchmark cases and tolerance policy before results are represented as validated.

Document control: Financial Risk Platform | Detailed Data Model | Working Draft v0.1 | Review with client data owners, security lead, financial-domain expert and solution architect before physical implementation.